Graphwise Stack — AWS EC2 + KIND

Maintainer: Kent Stroker

A **Helm-on-KIND** deployment of the Graphwise PoolParty ecosystem plus the GraphRAG chatbot suite, running on a single AWS EC2 instance (Amazon Linux 2023, Docker, single-node Kubernetes cluster). Designed as a personal demo environment for Graphwise field presales engineers; published MIT-licensed, AS-IS, no warranty, no support.

> **License files and credentials are NOT included.** Contact your Graphwise account team for `poolparty.key`, `graphdb.license`, `uv-license.key`, and Maven registry credentials. External users must supply their own AWS account, domain, and Graphwise licenses.

> **Demo-grade.** Default passwords, single-replica services, no HA, no hardening. Do not put real customer data in it.

Table of Contents

Where to start

I want to…	Go to…
Deploy as a PSE SE using the team kit	infra/terraform-subdomain/DEPLOYMENT_GUIDE.md
Understand what's running and why	This file (§ Architecture, § How a request gets routed)
See all URLs and credentials	This file (§ App URLs and credentials)
Understand the Terraform module and `user-data.sh.tpl`	infra/TERRAFORM_NOTES.md
Understand the chart internals, critical invariants, debug history	CLAUDE.md

---

What you get

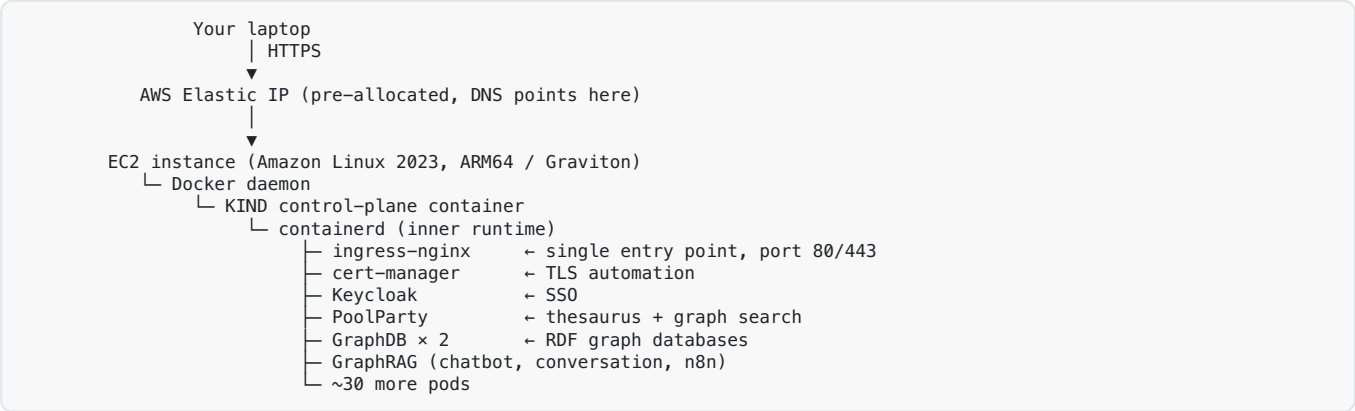
`terraform apply` provisions an EC2 host, bootstraps Docker + KIND + kubectl + helm via cloud-init, and brings up a single-node Kubernetes cluster. Two `helm install` runs put the entire Graphwise suite in the cluster:

- **PoolParty 10.3.0** — Thesaurus, GraphSearch, Extractor REST
- **GraphDB EE** (two instances: `embedded` for PoolParty, `projects` for federation demos)
- **Elasticsearch** (PoolParty's semantic search backend)
- **Keycloak** — SSO for PoolParty, ADF, Semantic Workbench, GraphRAG conversation
- **Addons** — ADF, Semantic Workbench, GraphViews, RDF4J, UnifiedViews, Ontotext Refine
- **GraphRAG** — chatbot, conversation API, components, n8n workflow engine
- **Console** — apex landing page with links to every app and mode toggle
- **Observability** — Kubernetes Dashboard, Prometheus, Grafana, AlertManager

Each app gets its own HTTPS subdomain and Let's Encrypt certificate. The whole thing costs ~$0.34/hr running and ~$30/mo while stopped (EBS + EIP retained).

Architecture

The stack in one breath



Host ports 80 and 443 are mapped into the KIND container in `infra/kind/kind-config.yaml`. Inside, ingress-nginx fans every request to the right app pod based on the URL's hostname. One door, dozens of rooms behind it.

How a browser request gets routed

```
Browser → DNS lookup → Elastic IP → EC2 port 443
       → KIND port 443 → ingress-nginx (TLS handshake, SNI lookup)
       → Kubernetes Service → App pod
```

Five hops. Troubleshooting works by asking "which hop broke?"

Symptom	Likely broken hop
DNS lookup fails / NXDOMAIN	DNS records missing or wrong IP
TCP timeout	Security Group, KIND port mapping, ingress-nginx pod not running
TLS error	Cert not issued, wrong `tls.secretName`, reflector not running
HTTP 502 / 503	App pod not Ready
OIDC redirect loop	Keycloak issuer mismatch (see CLAUDE.md § Critical rules)

Subdomain routing

Every app gets its own subdomain so each has its own LE cert and doesn't need context-path-prefix surgery. DNS requires exactly two records:

- `<sub>.<base>` → EIP (apex, for the Console landing page)
- `*.<sub>.<base>` → EIP (wildcard, for every app subdomain)

App	Hostname
Console (landing)	`<sub>.<base>`
Keycloak	`auth.<sub>.<base>`
PoolParty / GraphSearch / Extractor	`poolparty.<sub>.<base>`
GraphDB embedded	`graphdb.<sub>.<base>`
GraphDB projects	`graphdb-projects.<sub>.<base>`
ADF	`adf.<sub>.<base>`
Semantic Workbench	`semantic-workbench.<sub>.<base>`
GraphViews	`graphviews.<sub>.<base>`
RDF4J	`rdf4j.<sub>.<base>`
UnifiedViews	`unifiedviews.<sub>.<base>`
Ontotext Refine	`refine.<sub>.<base>` (CIDR-allowlisted)
GraphRAG chatbot / conversation / n8n	`graphrag.<sub>.<base>`
Kubernetes Dashboard	`dashboard.<sub>.<base>`
Prometheus	`prometheus.<sub>.<base>`
Grafana	`grafana.<sub>.<base>`

TLS — how Let's Encrypt certs happen automatically

cert-manager runs in the cluster and manages a single wildcard certificate (`<sub>.<base>` + `*.<sub>.<base>`) via the Let's Encrypt **DNS-01 challenge** against Route 53. The EC2 instance role (created by Terraform, scoped to the one hosted-zone ARN) lets cert-manager write the `_acme-challenge` TXT record without storing AWS credentials in the cluster.

Once issued, kubernetes-reflector copies the `wildcard-tls` Secret from the `cert-manager` namespace into every consuming namespace (`graphwise`, `graphrag`, `keycloak`, `kubernetes-dashboard`, `monitoring`). Every Ingress uses `tls.secretName: wildcard-tls`.

`letsencrypt-prod` **only.** The staging CA chain is not trusted by JVM clients (PoolParty, ADF, Semantic Workbench all talk to Keycloak over HTTPS in-cluster). A staging cert breaks OIDC for every app. LE prod's rate limit is 5 identical certs per 168 h — mitigate by using `pull-config.sh` before `terraform destroy` to save the live cert and `push-config.sh` after rebuild to restore it, skipping a DNS-01 round trip.

To check cert status: `kubectl get certificate -n cert-manager wildcard-tls` (want `READY=True`). Stuck in `False` → `kubectl describe order -n cert-manager` surfaces the AWS error. Most common cause: wrong `route53_zone_id` in `terraform.tfvars` (see CLAUDE.md resolved bug catalog).

Prerequisites

Required tools (one-time per laptop)

Tool	macOS	Windows
Package manager	Homebrew: `/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`	Chocolatey: run in admin PowerShell: `Set-ExecutionPolicy Bypass -Scope Process -Force; [System.Net.ServicePointManager]::SecurityProtocol = [System.Net.ServicePointManager]::SecurityProtocol -bor 3072; iex ((New-Object System.Net.WebClient).DownloadString('https://community.chocolatey.org/install.ps1'))`
AWS CLI	`brew install awscli`	`choco install awscli -y`
Terraform 1.5+	`brew install terraform`	`choco install terraform -y`
SSH	Ships with macOS	Enable via Settings → Optional Features → OpenSSH Client, or `choco install openssh -y`
`dig`	Ships with macOS	`choco install bind-toolsonly -y` (or use `nslookup` as fallback)
rsync	`brew install rsync` (macOS Ventura+ ships `openrsync` with compat quirks)	`choco install rsync -y`

Verify everything is installed:

```
git --version && aws --version && terraform version && ssh -V && dig -v && jq --version
```

AWS IAM setup (one-time per account, done by root or IAM admin)

This stack uses a **two-actor IAM model**. Never create IAM users with `terraform-demo`.

Actor 1 — `terraform-demo` (laptop, infra provisioning): - API access only (no Console login needed). - Attach AWS-managed `AmazonEC2FullAccess`. - Attach inline policy `graphwise-stack-iam` so Terraform can create/destroy the EC2 instance role + instance profile. Without it, `terraform apply` fails with `AccessDenied: iam:CreateRole`.

```
json { "Version": "2012-10-17", "Statement": [{ "Sid": "ManageEC2InstanceRole", "Effect": "Allow", "Action": [ "iam:CreateRole", "iam:DeleteRole", "iam:GetRole", "iam:UpdateRole", "iam:UpdateAssumeRolePolicy", "iam:PutRolePolicy", "iam:GetRolePolicy", "iam:DeleteRolePolicy", "iam:ListRolePolicies", "iam:ListAttachedRolePolicies", "iam:ListInstanceProfilesForRole", "iam:TagRole", "iam:UntagRole", "iam:ListRoleTags", "iam:CreateInstanceProfile", "iam:DeleteInstanceProfile", "iam:GetInstanceProfile", "iam:AddRoleToInstanceProfile", "iam:RemoveRoleFromInstanceProfile", "iam:TagInstanceProfile",
```

"iam:UntagInstanceProfile", "iam:PassRole"], "Resource": ["arn:aws:iam::*:role/graphwise-stack-*", "arn:aws:iam::*:instance-profile/graphwise-stack-*"] }] }

- Create an Access Key for use with `aws configure`.

Actor 2 — `graphrag-bedrock` (runtime, credentials stored in `graphwise-secrets.yaml`): - API access only. - Attach inline policy `bedrock-graphwise-invoke`. The inference-profile ARN is account-scoped — replace `<YOUR-ACCOUNT-ID>` with the 12-digit value from `aws sts get-caller-identity`.

json { "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Action": ["bedrock:InvokeModel", "bedrock:InvokeModelWithResponseStream"], "Resource": ["arn:aws:bedrock:us-west-2::foundation-model/amazon.titan-embed-text-v2:0", "arn:aws:bedrock:us-east-1::foundation-model/meta.llama3-3-70b-instruct-v1:0", "arn:aws:bedrock:us-east-2::foundation-model/meta.llama3-3-70b-instruct-v1:0", "arn:aws:bedrock:us-west-2::foundation-model/meta.llama3-3-70b-instruct-v1:0", "arn:aws:bedrock:us-west-2:<YOUR-ACCOUNT-ID>:inference-profile/us.meta.llama3-3-70b-instruct-v1:0"] }] }

Titan embed powers the GraphRAG embedding call; Llama 3.3 cross-region profile powers PoolParty's Taxonomy Advisor. If your Bedrock region isn't `us-west-2`, swap every `us-west-2` ARN above (including the inference-profile ARN). Want a different LLM? Swap the Llama ARNs for Nova Pro, Nova Lite, or Mistral Large — all gate-free. Anthropic Claude requires a one-time use-case form in the Bedrock Console.

- Create an Access Key → paste into `graphwise-secrets.yaml` under `graphrag-secrets.awsCredentials`.

Bedrock model access gates: - Amazon Titan, Meta Llama, Mistral: gate-free — no approval form needed. - Anthropic Claude: requires a one-time use-case form in the Bedrock Console (5–15 min approval). The stack defaults to Llama 3.3 to avoid this gate. - Newer models (Llama 3.3+, Claude 3.5 v2+, Nova, Mistral Large 2) require an **inference profile ID** (`us.` / `eu.` / `apac.` prefix); bare foundation-model IDs return `InvalidRequestException`.

Configure the Terraform actor:

```
aws configure           # paste terraform-demo key + secret + region + json output
aws sts get-caller-identity     # must show user/terraform-demo
```

EC2 key pair

AWS Console → EC2 → Network & Security → Key Pairs → Create key pair → RSA, `.pem` format → download → `chmod 400 ~/.ssh/<name>.pem`.

Pre-allocate Elastic IP

Allocate **before** `terraform apply` so the IP is known at DNS-record creation time:

```
aws ec2 allocate-address --domain vpc --region <region>
# Save both AllocationId (eipalloc-...) and PublicIp
```

DNS records

Two A records, both pointing at the pre-allocated IP:

Name	Value	TTL
`<sub>.<base>`	EIP	300
`*.<sub>.<base>`	EIP	300

Propagation is usually under 5 minutes. Verify before proceeding:

```
dig +short <sub>.<base> poolparty.<sub>.<base>
# Both lines should return the EIP
```

EC2 Instance Connect (optional, macOS)

```
# Attach ec2-instance-connect:SendSSHPublicKey inline policy to terraform-demo (IAM admin step):
# { "Effect": "Allow", "Action": "ec2-instance-connect:SendSSHPublicKey",
#   "Resource": "arn:aws:ec2:*:*:instance/*",
#   "Condition": { "StringEquals": { "ec2:osuser": "ec2-user" } } }

# Then connect:
aws ec2-instance-connect ssh --instance-id <instance-id> --private-key-file ~/.ssh/<key>.pem
```

The browser-based console connection requires a manual SG inbound rule for the `com.amazonaws.<region>.ec2-instance-connect` prefix list. This rule is added manually in the Console (not managed by Terraform — it survives future applies because the SG carries `ignore_changes = [ingress]`).

Provisioning and bootstrap

```
cd infra/terraform-<stack>
cp terraform.tfvars.example terraform.tfvars
$EDITOR terraform.tfvars          # fill in REQUIRED block
terraform init
terraform plan                    # READ this; ~5 resources to create
terraform apply                   # ~3-5 min for AWS; cloud-init runs ~2-3 min more
```

Watch the bootstrap:

```
ssh -i ~/.ssh/<key>.pem ec2-user@<host> 'sudo tail -f /var/log/bootstrap.log'
# Wait for "=== Bootstrap complete ===" then Ctrl-C
```

Immediately after first apply — lock the AMI:

```
terraform output -raw ami_id      # prints ami-...
$EDITOR terraform.tfvars          # add: ami_override = "ami-..."
terraform plan                    # MUST show "No changes"
```

This prevents a future AWS-published AL2023 refresh from force-replacing your EC2. See infra/TERRAFORM_NOTES.md → Safety for full rationale.

Set up SSH convenience entries (optional but recommended):

```
cd infra/terraform-subdomain
./scripts/manage-stacks.sh add    # prompts for name, host, key, alias
source ~/.zprofile                # activate the alias immediately
```

After that, `ssh<name>` drops you in as `ec2-user`. For scp:

```
./scripts/stack-scp.sh logo.png :~/logo.png          # push to EC2
./scripts/stack-scp.sh :~/wildcard-tls.yaml ./       # pull from EC2
./scripts/stack-scp.sh -r ./data :~/staging-data/    # push recursively
```

For the full post-apply build sequence, see DEPLOYMENT_GUIDE.md.

Operator secrets and credentials

All operator-supplied secrets live in `~/graphwise-secrets.yaml` on the EC2 — auto-created by Terraform cloud-init, gitignored, never tracked. `reset-helm.sh` auto-includes it via `-f`. Editing this file (instead of chart values) means `git pull` is always a clean fast-forward with no merge conflict against your keys.

```yaml
# ~/graphwise-secrets.yaml (EC2-local, never committed)
maven:
  user: ""        # FILL IN: Graphwise Maven user
  pass: ""        # FILL IN: Graphwise Maven password

graphrag-secrets:
  awsCredentials:
    region: "us-west-2"
    accessKeyId: ""       # FILL IN: graphrag-bedrock AKIA...
    secretAccessKey: ""   # FILL IN
  n8nLicense:
    activationKey: ""     # FILL IN: n8n Enterprise key
  n8nEncryption:
    key: "..."            # AUTO-GENERATED by Terraform — do NOT change this
```

`n8nEncryption.key` is immutable. Changing it makes every saved n8n connection credential unreadable with no recovery path. It is generated once by Terraform `random_id` and stays constant across all `helm upgrade` runs.

License files go to `~/gsb/files/licenses/` with these exact names (vendor files arrive with engagement-specific names — rename them):

```
files/licenses/poolparty.key
files/licenses/graphdb.license
files/licenses/uv-license.key
```

The pull/push-config cycle (survive terraform destroy)

Before `terraform destroy`, save the operator state from the live EC2:

```
cd infra/terraform-<stack>
./scripts/pull-config.sh          # saves to ./graphwise-config-<host>-<UTC>/
```

Captures: `graphwise-secrets.yaml`, license files, live wildcard TLS cert, dashboard kubeconfig. After the next `terraform apply`, restore everything in one shot:

```
cd infra/terraform-<stack>
./scripts/push-config.sh          # auto-discovers the most recent snapshot
```

The TLS cert restoration is the headline: `cluster-bootstrap.sh` detects `~/wildcard-tls-saved.yaml` and applies the saved cert before running any DNS-01 challenge, preserving a Let's Encrypt weekly rate-limit slot.

Day-2 lifecycle

Polite EC2 stop/start

```
# EC2 — quiesce app workloads before stopping
./scripts/cluster-stop.sh
# Records each workload's replica count in a k8s annotation for auto-restore.

# Laptop — stop the EC2 (~$0.34/hr running → ~$30/mo stopped)
aws ec2 stop-instances --instance-ids <instance-id> --region <region>

# Laptop — start it back
aws ec2 start-instances --instance-ids <instance-id> --region <region>
# The graphwise-cluster-resume.service systemd unit fires on every EC2 start,
# restarts KIND node containers, and calls cluster-start.sh to scale workloads
# back to their pre-stop counts. No operator action needed after start.
```

Wipe and reinstall (destroys all app data)

```
# EC2
./scripts/reset-helm.sh --yes <subdomain>          # both releases, ~10-15 min
./scripts/reset-helm.sh --yes --skip-graphrag <sub> # umbrella only, ~7-10 min
```

Non-destructive chart upgrade

```
# EC2
helm upgrade graphwise-stack ./charts/graphwise-stack -n graphwise \
    -f charts/graphwise-stack/values.yaml \
    -f $HOME/.graphwise-stack/values-<sub>.yaml \
    -f ~/graphwise-secrets.yaml --timeout 15m

helm upgrade graphrag ./charts/vendor/graphrag -n graphrag \
    -f charts/vendor/graphrag/values-graphwise.yaml \
    -f $HOME/.graphwise-stack/values-<sub>-graphrag.yaml --timeout 15m
```

Secret-only changes (AWS keys, Maven creds updated in `graphwise-secrets.yaml`) require a manual rollout restart — `secretKeyRef` env vars are snapshotted at pod start:

```
kubectl -n graphwise rollout restart deploy/graphwise-stack-poolparty
kubectl -n graphrag rollout restart deploy/graphrag-chatbot-api
```

AMI-based multi-stack management

For PSE SEs managing multiple named customer stacks (Oil & Gas, VA Benefits, etc.) a golden-AMI workflow avoids re-running the 25-minute build for every demo:

Day-0 (build the AMI):

```
# After a fully-validated reset-helm, on EC2:
./scripts/cluster-stop.sh
# Laptop: AWS Console → EC2 → right-click instance → Create Image → name it
# Record the ami-... value, set ami_override in terraform.tfvars
# Test the destroy/apply cycle: terraform destroy && terraform apply
```

Day-1 (spin up from AMI):

```
terraform apply          # ~3 min; graphwise-cluster-resume.service auto-restores
```

Day-2 (maintenance): - TLS cert renews ~60 days after issue. Re-snapshot the AMI after renewal so the next `terraform apply` doesn't cold-start cert issuance. - Chart/image upgrades: `reset-helm.sh` on the running instance → validate → snapshot. - AMI recovery: if an AMI-booted instance is broken, comment out `ami_override` in `terraform.tfvars` and `terraform apply` — falls back to the latest AL2023 stock.

Terraform safety rule: never run unscoped `terraform apply` post-provision without reading `terraform plan` character by character. The AMI data source (`most_recent = true`) can trigger a force-replace. See infra/TERRAFORM_NOTES.md → Safety.

Customer logo branding

```
# Laptop — scp a PNG (transparent background recommended)
scp -i $GRAPHWISE_KEY ~/logo.png $GRAPHWISE_USER@$GRAPHWISE_HOST:~/logo.png

# EC2
cd ~/gsb
./scripts/set-logo.sh ~/logo.png   # base64-encodes → gitignored console-branding.yaml
./scripts/reset-helm.sh --yes <sub>
```

Uploading ingest data

Cloud-init creates `~/staging-data/` as a landing pad. Use rsync for large uploads:

```
# Laptop (install rsync if missing: brew install rsync)
rsync -azP -e "ssh -i $GRAPHWISE_KEY" ~/local-pdfs/ $GRAPHWISE_USER@$GRAPHWISE_HOST:~/staging-data/

# Fallback (no resume if interrupted)
scp -r -i $GRAPHWISE_KEY ~/local-pdfs/ $GRAPHWISE_USER@$GRAPHWISE_HOST:~/staging-data/
```

Staging data survives EC2 stop/start and `reset-helm.sh` but **not** `terraform destroy`.

The stack wires a three-layer path from EC2 disk to pod filesystem: `~/staging-data/` (EC2 EBS) → KIND `extraMount` (`/staging-data` inside the container) → Kubernetes `hostPath` PV → PVC `staging-data` in each namespace → pod `volumeMount`. See CLAUDE.md → Chart internals for the PV/PVC YAML and checklist.

App URLs and credentials

Replace `<sub>.<base>` with your deployment's apex hostname (e.g. `kstroker.gw-pse.com`).

> Default password pattern: `rdf#rocks` for most app logins, basic-auth, and Postgres passwords. Exceptions are noted below.

App	URL	Username	Password
Console landing	`https://<sub>.<base>/`	—	—
Keycloak Admin Console	`https://auth.<sub>.<base>/admin/`	`poolparty_auth_admin`	`admin`
PoolParty Thesaurus	`https://poolparty.<sub>.<base>/PoolParty/`	`superadmin`	`poolparty`
GraphSearch	`https://poolparty.<sub>.<base>/GraphSearch/`	SSO via Keycloak	—
ADF	`https://adf.<sub>.<base>/ADF/`	SSO via Keycloak	—
Semantic Workbench	`https://semantic-workbench.<sub>.<base>/SemanticWorkbench/`	SSO via Keycloak	—
GraphViews	`https://graphviews.<sub>.<base>/GraphViews/`	direct API auth	—
GraphDB embedded	`https://graphdb.<sub>.<base>/`	`demo`	`rdf#rocks`
GraphDB projects	`https://graphdb-projects.<sub>.<base>/`	`demo`	`rdf#rocks`
RDF4J Workbench	`https://rdf4j.<sub>.<base>/rdf4j-workbench/`	`demo`	`rdf#rocks`
Ontotext Refine	`https://refine.<sub>.<base>/`	CIDR-allowlisted (no login)	—
GraphRAG chatbot	`https://graphrag.<sub>.<base>/`	`alice` or `bob`	`alice123` / `bob123`
GraphRAG Conversation API	`https://graphrag.<sub>.<base>/conversations/`	OIDC bearer token	—
GraphRAG Workflows (n8n)	`https://graphrag.<sub>.<base>/graphrag/workflows/`	set on first visit	—
Kubernetes Dashboard	`https://dashboard.<sub>.<base>/`	Kubeconfig upload	`~/dashboard-kubeconfig.yaml`
Prometheus	`https://prometheus.<sub>.<base>/`	`demo`	`rdf#rocks`
Grafana	`https://grafana.<sub>.<base>/`	`admin`	`demo-graphwise-2026`
UnifiedViews	`https://unifiedviews.<sub>.<base>/UnifiedViews/`	`admin`	`admin`

Kubernetes Dashboard paste bug: Chrome and Safari silently drop the token when pasting into the Dashboard login field (v2.7.0 paste-handler issue). Use the **Kubeconfig upload** path instead. Retrieve the kubeconfig from the EC2:

```
scp -i $GRAPHWISE_KEY $GRAPHWISE_USER@$GRAPHWISE_HOST:~/dashboard-kubeconfig.yaml ~/Downloads/
```

Then on the Dashboard login screen, switch the radio to **Kubeconfig** and upload the file.

Activating PoolParty "Build Your Taxonomy" (LLM feature)

The chart wires the Bedrock + Llama 3.3 70B backend at deploy time, but the feature is gated by a per-deployment **Taxonomy Advisor** instance you create once:

1. Request the Taxonomy Advisor API key from your Graphwise contact.
2. Log in to PoolParty → Semantic Middleware Configurator (SMC).
3. Expand External Services → double-click **Taxonomy Advisor** → Create instance.
4. Enter a name and paste the API key → Save.

If IAM is wrong: `kubectl -n graphwise logs deploy/graphwise-stack-poolparty | grep -iE 'bedrock|accessdenied'`. Common errors: `AccessDeniedException` (IAM policy missing inference-profile ARN), `InvalidRequestException … on-demand throughput` (used bare foundation-model ID — must use `us.<model-id>` inference profile), `ResourceNotFoundException … use case details` (Anthropic Claude requires the Bedrock Console use-case form).

Helm releases overview

Two separate releases — order matters on both install and uninstall:

Release	Namespace	Chart	Install first?
`graphwise-stack`	`graphwise`	`charts/graphwise-stack/` (umbrella)	Yes — creates Secrets/Postgres in `graphrag`
`graphrag`	`graphrag`	`charts/vendor/graphrag/`	No — needs umbrella's Secrets to exist first

`scripts/reset-helm.sh` enforces the ordering for both install and uninstall.

The umbrella chart installs `charts/graphdb/` **twice** via aliases (`graphdb-embedded` in `graphwise` ns, `graphdb-projects` in `graphdb` ns) — giving two fully independent GraphDB instances from one chart definition.

For chart internals, invariants that cause stack breakage, and the full resolved bug catalog, see CLAUDE.md.

Keycloak SSO and the OIDC issuer invariant

PoolParty, ADF, Semantic Workbench, and the GraphRAG conversation API all authenticate via Keycloak OIDC. Spring Security's `NimbusJwtDecoder.withIssuerLocation()` does a **strict byte-for-byte equality check** on the `iss` claim in every JWT. All OIDC clients must be configured with exactly:

```
https://auth.<sub>.<base>/realms/<realm>
```

No `/auth` path prefix, no trailing slash. The Keycloak CR must have `spec.hostname.hostname: auth.<sub>.<base>` and `strict: true`.

Quick issuer check:

```
curl -s https://auth.<sub>.<base>/realms/poolparty/.well-known/openid-configuration \
  | jq -r .issuer
# Must print exactly: https://auth.<sub>.<base>/realms/poolparty
```

Troubleshooting

Quick-reference runbook

Symptom	First command	Resolution path
TLS error on any URL	`kubectl get certificate -n cert-manager wildcard-tls`	`READY=False` → `kubectl describe order -n cert-manager`. AccessDenied = wrong `route53_zone_id`.
Wildcard cert stuck `False`	`kubectl describe challenge -n cert-manager`	AccessDenied on Route 53 → see CLAUDE.md bug catalog for the `aws iam put-role-policy` fix.
Browser hangs	`dig +short <host>`	DNS must resolve to EIP. If wrong, fix DNS first.
`kubectl` refuses connection after EC2 reboot	`./scripts/cluster-resume.sh`	KIND node containers stopped. Resume restarts them + pins `--restart=unless-stopped`.
Pod in `0/1 Running` for >2 min	`kubectl describe pod -n <ns> <pod>` then `kubectl logs ...`	`describe` shows probe failures and events; `logs` shows the app's own error.
OIDC redirect loop	`curl ... .well-known/openid-configuration | jq -r .issuer`	Issuer must match `https://auth.<apex>/realms/<realm>` exactly.
PoolParty `Internal Error` after Keycloak login	`kubectl get job -n keycloak | grep authz-import`	The authz-import Job restores per-client authorization settings. If `Failed`, re-run manually.
`ImagePullBackOff` on graphrag pods	`kubectl describe pod ... | grep -A5 'Failed to pull'`	Maven creds wrong or empty in `graphwise-secrets.yaml`. Re-upgrade with all `-f` flags.
`terraform apply` wants to replace EC2	STOP — read `terraform plan` fully	AMI force-replace. Ensure `ami_override` is set and `lifecycle.ignore_changes` is in `main.tf`.

SSH session predates rebuild

If you SSH in before `terraform apply` has fully updated `/etc/profile.d/graphwise.sh`, `cluster-bootstrap.sh` fails immediately at the `${ROUTE53_ZONE_ID:?}` check after installing only ingress-nginx. Fix:

```
source /etc/profile.d/graphwise.sh   # in the current SSH session
```

The script is idempotent — safe to re-run from the point of failure.

PoolParty LLM check

```
kubectl -n graphwise exec deploy/graphwise-stack-poolparty -- env | grep -E '^POOLPARTY_LLM|^AWS_'
```

If `AWS_ACCESS_KEY_ID` is empty, a `helm upgrade` ran without `-f ~/graphwise-secrets.yaml`. Re-upgrade with all three `-f` flags, then rollout-restart the deployment.

Grafana password rotation

Edit `charts/observability/kube-prometheus-stack-values.yaml` → `grafana.adminPassword`, then re-run `cluster-bootstrap.sh`.

n8n workflow engine notes

- n8n DB is restored from a workflow dump placed in the EC2 home root as `$HOME/workflows-pg-dumpall-<date>-v<N>.sql` (scp'd up by the operator, or produced on the box by `scripts/create-workflows-dumpall.sh`); `scripts/restore-workflows-dumpall.sh` loads the NEWEST such file into the CNPG Postgres cluster. No dump is shipped in the repo/clone — if none is present the restore is a no-op (n8n starts empty).
- When restoring from a `pg_dumpall` dump, `DROP DATABASE n8n WITH (FORCE)` first — the dump has no `--clean` clause so old artifacts survive if you don't. `CREATE ROLE postgres/streaming_replica` lines error harmlessly (CNPG-managed roles already exist). Re-assert `ALTER ROLE n8n PASSWORD 'rdf#rocks'` + public grants after.
- The n8n **Configuration** workflow's `poolPartyProjectId` and GraphDB paths must be updated for your deployment or every ingest workflow fails.

Repo layout

```
charts/
  graphwise-stack/        Umbrella Helm chart (PoolParty, GraphDB ×2, ES, console,
                           addons, Keycloak, graphrag supporting Secrets/Postgres)
  poolparty/              PoolParty sub-chart
  graphdb/                GraphDB sub-chart (installed twice via aliases)
  addons/                 ADF, Semantic Workbench, GraphViews, RDF4J, UnifiedViews, Refine
  console/                Apex landing page
  poolparty-elasticsearch/ Elasticsearch for PoolParty
  keycloak-realms/        KeycloakRealmImport CRs + post-install Jobs
  vendor/graphrag*/       Vendored GraphRAG charts (separate Helm release)

infra/
  kind/kind-config.yaml   Single-node KIND cluster definition (host port mappings,
                           extraMounts for staging-data)
  terraform-subdomain/    Terraform module: EC2, SG, IAM role, EIP association,
                           cloud-init bootstrap, AMI data source, EIP attachment
    scripts/
      check-prereqs.sh    macOS preflight: tools, AWS CLI identity, DNS, SSH key
      manage-stacks.sh    Add/list/remove GW stack SSH entries in ~/.zprofile as
                           sentinel-delimited blocks (GW_KEY_<name>, GW_HOST_<name>,
                           ssh alias). Subcommands: list, add, remove, interactive menu.
      stack-scp.sh        scp wrapper that auto-fills -i key and ec2-user@host from
                           ~/.zprofile blocks; prefix EC2 paths with ':'
      pull-config.sh      Snapshot operator secrets + licenses + wildcard TLS cert
                           from live EC2 into a dated folder in the current directory
      push-config.sh      Restore a pull-config.sh snapshot to a freshly-provisioned EC2

scripts/
  cluster-bootstrap.sh    One-time: install ingress-nginx, cert-manager, CNPG,
                           Keycloak operator, metrics-server, Dashboard, kube-prometheus
  cluster-resume.sh       Restart KIND after EC2 stop/start (also invoked by systemd)
  cluster-stop.sh         Scale-to-0 app workloads before EC2 stop
  cluster-start.sh        Restore replica counts from annotations (used by cluster-resume)
  render-values.sh        Emit per-subdomain values YAML into ~/.graphwise-stack/
  reset-helm.sh           Wipe + reinstall both Helm releases
  deploy-stack.sh         Non-interactive chain: bootstrap → realm extract → licenses
                           → preflight → reset-helm → validate (PSE kit builds)
  install-licenses.sh     Create K8s Secrets from files/licenses/
  extract-poolparty-realm.sh   Pull PoolParty realm JSON from Keycloak image +
                               substitute Ontotext placeholder variables
  preflight-reset-helm.sh Read-only gate: tools, cluster, operators, DNS, IMDS,
                           maven registry auth
  validate-bootstrap.sh   Post-bootstrap health check
  validate-stack.sh       Post-reset-helm health check (pods, certs, OIDC, HTTPS)
  restore-workflows-dumpall.sh  Load workflow DB from newest $HOME/workflows-pg-dumpall*.sql
  create-workflows-dumpall.sh   Snapshot live workflow DB -> $HOME/workflows-pg-dumpall-<date>.sql
  check-image-versions.sh Check/upgrade image tags vs Docker Hub; --apply rolls the live stack
  set-logo.sh             Base64-encode a PNG → gitignored console-branding.yaml
  build-refine-image.sh   Build multi-arch Refine image from bundled dist

files/
  licenses/               Gitignored vendor license binaries (poolparty.key,
                           graphdb.license, uv-license.key)
  refine/ontorefine-1.2.1/ Bundled platform-independent Ontotext Refine dist
                           (amd64-only upstream image; this avoids the ARM64 crash)
```

External user notes

This repo is MIT-licensed but ships without credentials or license files.

1. **Domain** — you need a domain whose DNS is hosted in Route 53 (so cert-manager can write the `_acme-challenge` TXT records). Transfer NS delegation to a Route 53 hosted zone if the domain is registered elsewhere.
2. **AWS account** — any account works. Provision the two IAM users per § Prerequisites → AWS IAM setup.

3. **Graphwise licenses** — contact `support@graphwise.ai` or your account team for `poolparty.key`, `graphdb.license`, and `uv-license.key`. Also request Maven registry credentials (`maven.user` / `maven.pass`) for the `graphdb`, `poolparty`, and GraphRAG image pulls.
4. **n8n Enterprise** — n8n AI features (GraphRAG workflows) require an n8n Enterprise license key. Trial available at n8n.io.
5. **Subdomain** — pick any subdomain under your domain. Update `terraform.tfvars` accordingly.

Appendix: `scripts/` reference

Every operational script lives under `scripts/` and runs **on the EC2 host** (as `ec2-user`). Many are chained automatically — `deploy-stack.sh` runs the full build sequence end-to-end, and the `graphwise-cluster-resume.service` systemd unit runs `cluster-resume.sh` on every boot — so the ones marked *(auto)* / *(boot)* are rarely invoked by hand.

> The laptop-side kit helpers — `check-prereqs.sh`, `pull-config.sh`, `push-config.sh`, `manage-stacks.sh`, `stack-scp.sh` — live under `infra/terraform-example/scripts/` and are documented in the kit's `README.pdf`, not here.

Summary table

Script	Runs on	Purpose
Provisioning & deploy		
`deploy-stack.sh`	EC2	One-shot non-interactive build; chains bootstrap → realm/licenses → reset-helm → workflow restore
`cluster-bootstrap.sh`	EC2	Install cluster operators + observability + wildcard cert; pre-load Refine/ Keycloak images
`reset-helm.sh`	EC2	Wipe (incl. PVCs) and reinstall both Helm releases from scratch
`render-values.sh`	EC2 *(auto)*	Emit the two per-subdomain Helm values overlays into `~/.graphwise-stack/`
Licenses, secrets & realm		
`install-licenses.sh`	EC2	Create the three vendor license Secrets from `files/licenses/`
`extract-poolparty-realm.sh`	EC2	Pull + placeholder-substitute the PoolParty Keycloak realm JSON
Preflight & validation		
`preflight-reset-helm.sh`	EC2	Read-only gate that checks every precondition before the destructive reset-helm
`validate-bootstrap.sh`	EC2	Post-bootstrap operator/issuer/secret health check
`validate-stack.sh`	EC2	Post-reset-helm workload / cert / OIDC / HTTPS health check
Day-2 lifecycle		
`cluster-stop.sh`	EC2	Scale app workloads to 0 (recording replica counts) before stopping the EC2
`cluster-start.sh`	EC2 *(auto)*	Restore workloads to their pre-stop replica counts
`cluster-resume.sh`	EC2 *(boot)*	Restart KIND node containers after a reboot, then start workloads
Workflow database seed		
`create-workflows-dumpall.sh`	EC2	Snapshot the live workflow DB to `$HOME/workflows-pg-dumpall-<date>.sql`
`restore-workflows-dumpall.sh`	EC2	Load the newest `$HOME/workflows-pg-dumpall*.sql` into n8n Postgres
`register-n8n-api-key.sh`	EC2	Register the seed's n8n public-API key so API-calling nodes don't 401
Branding, images & utilities		
`set-logo.sh`	EC2	Base64-encode a customer logo into a gitignored console-branding overlay
`check-image-versions.sh`	EC2	Check image tags vs Docker Hub, upgrade charts, and (`--apply`) roll the live stack in place

Script	Runs on	Purpose
`build-refine-image.sh`	EC2	Build a multi-arch Refine image from the bundled dist and load it into KIND

Provisioning & deploy

`deploy-stack.sh`

The one-shot, non-interactive build for a brand-new stack — what the PSE kit's `terraform-deploy.sh` triggers. It chains, in order: `cluster-bootstrap.sh` → `extract-poolparty-realm.sh` (which itself chains `install-licenses.sh`) → `reset-helm.sh --yes` (umbrella first, then graphrag) → `restore-workflows-dumpall.sh`. Use this for a clean full build; reach for the individual scripts below only when troubleshooting a specific stage.

`cluster-bootstrap.sh`

"Phase B" — installs the cluster operators and prerequisites into the single-node KIND cluster that cloud-init created: ingress-nginx, cert-manager + the Let's Encrypt `ClusterIssuer`, CNPG, the Keycloak operator, metrics-server, the Kubernetes Dashboard, and kube-prometheus-stack. It also mints the wildcard `Certificate` and pre-loads the Refine + PoolParty-Keycloak images into KIND. Run once after the cluster is up and DNS points at the EIP — it does **not** block on DNS, but no `Certificate` goes Ready until DNS resolves. Prereq: `~/graphwise-secrets.yaml` with `maven.user` / `maven.pass`. Idempotent.

`reset-helm.sh`

The troubleshooting workhorse: wipes and reinstalls **both** Helm releases from scratch — including PVCs, so you start from blank data — then `helm upgrade --install`s the umbrella first and graphrag second. Re-renders the per-subdomain overlay first (auto-invokes `render-values.sh`). Leaves the cluster operators and the KIND cluster untouched. Flags: `--yes` (skip the destructive-action confirm), `--skip-graphrag`. For a change that shouldn't nuke data, use `helm upgrade` directly instead.

`render-values.sh`

Given the subdomain, emits the two Helm values overlays this stack needs — `values-<sub>.yaml` (umbrella) and `values-<sub>-graphrag.yaml` (GraphRAG) — into `~/.graphwise-stack/`. **You usually don't run this by hand**; `reset-helm.sh` calls it twice. It also auto-emits the Refine local-image override when the bundled dist is detected.

Licenses, secrets & realm

`install-licenses.sh`

kubectl-creates the three vendor license Secrets — `poolparty-license`, `graphdb-license`, `unifiedviews-license` — from `files/licenses/*` (vendor blobs that never enter git; scp them from your laptop first). Run after `cluster-bootstrap.sh` and before installing the umbrella. Idempotent — re-runs replace the Secrets in place, and charts pick up new license content on the next pod restart.

`extract-poolparty-realm.sh`

Pulls the PoolParty Keycloak realm JSON out of the `ontotext/poolparty-keycloak` image, substitutes the Ontotext placeholder variables, and drops it where the `keycloak-realms` chart expects it. The JSON holds client secrets and password hashes, so it's gitignored. Re-run whenever you bump the poolparty-keycloak image version. (Chains `install-licenses.sh`.)

Preflight & validation

`preflight-reset-helm.sh`

A read-only gate that checks every precondition `reset-helm.sh` needs **before** the destructive uninstall: tools, cluster, operators, DNS, IMDS, maven registry auth, license/realm presence, and credential completeness. Its job is to catch ImagePullBackOff, "license not found", LE cert failures, and bad credentials before Helm spends 10–15 minutes installing pods that are doomed to crash. Flags: `--skip-graphrag`, `--strict` (warnings become failures). Safe to run repeatedly.

`validate-bootstrap.sh`

Post-`cluster-bootstrap` health check: walks every operator namespace, the `ClusterIssuer`, the image-pull secrets, and the dashboard kubeconfig, then prints a per-check pass/fail summary and an overall verdict. Read-only and idempotent. Exit `0` = all pass, `1` = one or more failed.

`validate-stack.sh`

Post-`reset-helm` health check: workloads across every namespace, the Helm releases, license + image-pull secrets, the GraphDB rename, staging-data PVCs, the Keycloak post-install Jobs, every `Certificate`, the OIDC issuer match for all three realms, and an HTTPS reachability sweep of every app URL. Closes with a "where to click next" panel. Read-only; needs `GRAPHWISE_APEX` (auto-set by cloud-init).

Day-2 lifecycle scripts

`cluster-stop.sh`

Politely quiesces the stack before you stop the EC2: scales every Deployment + StatefulSet in the `graphwise` / `graphrag` namespaces to 0 (recording each workload's prior replica count in the `graphwise.ai/replicas-before-stop` annotation), waits for pods to terminate, and prints the AWS stop command. It does **not** stop the instance itself (that would kill your SSH session) and leaves the operator namespaces alone.

`cluster-start.sh`

The symmetric partner to `cluster-stop.sh`: reads the `replicas-before-stop` annotation back, scales each workload to its recorded count, and clears the annotation. Annotation-gated, so it restores **only** what `cluster-stop.sh` scaled and leaves deliberately-disabled workloads untouched. Auto-invoked at the end of `cluster-resume.sh`.

`cluster-resume.sh`

After an EC2 stop/start the KIND node containers stay `Exited` (no restart policy), so kubectl fails with "connection refused". This restarts them, sets `restart=unless-stopped` so future reboots are a non-event, waits for the kube API to answer, then calls `cluster-start.sh` to scale workloads back up. Run automatically on every boot by the `graphwise-cluster-resume.service` systemd unit, so the stack survives a reboot hands-off. `--if-exists` makes it a no-op when no cluster is present.

Workflow database seed

`create-workflows-dumpall.sh`

Snapshots the live workflow (n8n) database with `pg_dumpall` into `$HOME/workflows-pg-dumpall-<date>.sql` on the EC2 home root. Because `restore-workflows-dumpall.sh` loads the *newest* such file, a fresh dump created here is what the next restore picks up. Run after the stack is up (Postgres + n8n Running).

`restore-workflows-dumpall.sh`

Loads a workflow DB seed into the freshly-initdb'd n8n Postgres: drops the existing DB, loads the newest `$HOME/workflows-pg-dumpall*.sql` (by date + version sort), then re-asserts the n8n role password and public grants. The seed is **not** shipped in the repo — scp one up or produce it with `create-workflows-dumpall.sh`; if none is present the script is a safe no-op. Called as the last step of `deploy-stack.sh`.

`register-n8n-api-key.sh`

Registers the seed's n8n public-API key in `public.user_api_keys` so nodes that call the n8n public API (token-usage calc, execution loaders) don't 401. The JWT ships in the seed's API-keys data table, but the registration row came back empty; this inserts the matching registration, tied to the owner user. Idempotent (guarded by `WHERE NOT EXISTS`).

Branding, images & utilities

`set-logo.sh`

Base64-encodes a customer logo PNG into a persistent, gitignored Helm overlay (`$HOME/.graphwise-stack/console-branding.yaml`) so the console landing-page header shows it; `reset-helm.sh` includes the overlay automatically when present. Run with `--clear` to remove branding and revert to the unbranded header. Usage: `scripts/set-logo.sh <image-file> [customer-name]`.

`check-image-versions.sh`

The in-place stack updater. Reads the current image tag from every chart values file, fetches the latest published semver tag for each image from Docker Hub, prints a comparison table, and offers to upgrade each outdated image interactively — editing the chart values under `~/gsb/charts` and rebuilding the umbrella's bundled tarballs (`helm dependency update`). With `--apply` it then rolls the *running* stack to the new images **without a destroy**: for each accepted upgrade it `docker pull`s the new tag and `kind load`s it into the cluster, then does a non-destructive `helm upgrade` of the `graphwise-stack` (umbrella) release — every

catalogued image lives there — reusing the deployment's existing values overlays, so PVCs (and data) are retained. Without `--apply` it only edits the charts (re-run with `--apply`, or commit the bump). Flags: `--yes` (accept all), `--apply` (roll the live stack), `--timeout <dur>` (helm upgrade timeout, default 15m). Runs on the EC2; needs `curl` + `jq` (plus `helm` / `docker` / `kind` / `kubectl` for `--apply`).

`build-refine-image.sh`

Wraps the platform-independent Ontotext Refine ZIP (a pure-Java app, no native binaries) in an arm64-compatible JRE container and loads it into KIND as `graphwise-refine:local`. It exists because `ontotext/refine:1.2.x` on Docker Hub is amd64-only while the canonical deploy is AL2023 Graviton (arm64). The ZIP is gitignored (~330 MB vendor binary) — extract it once under `refine/ontorefine-1.2.1/`. Idempotent, and auto-run by `cluster-bootstrap.sh`.